Assignment 2 – Problem 1

Learning Word Embeddings from IIT Jodhpur Data

Name : Aryan Baranwal
Roll No. : M25CSE035
Course: Natural Language Understanding

1. Introduction

Dense and high-dimensional representations of words as vectors representing semantic connections between words are called word embeddings. Word embeddings is in contrast to traditional one-hot encoding techniques in that words with similar meaning can share similar vector representations in a continuous vector space. The given assignment is aimed at constructing and examining word embeddings based on textual data gathered by visiting several IIT Jodhpur web pages. The project includes:

- Scraping textual material from IIT Jodhpur sources
- Preprocessing and cleaning of the collected data.
- Word2Vec model training on two architectures.
- Implementing Word2Vec from scratch
- Comparison of results with Gensim's implementation
- Evaluating the embeddings using nearest neighbours and analogy tasks
- Instructions visualizing the learned embeddings using PCA and t-SNE

It is aimed at knowing how words in text data have acquired semantic relationships between them through context.

2. Dataset Preparation

2.1 Data Sources

Textual data was collected from multiple pages of the **IIT Jodhpur website**. The sources include:

1. Academic regulations
2. Department overview pages (CSE, EE, ME, etc.)
3. Faculty profile pages
4. Academic program descriptions (BTech, MTech, PhD)
5. General institute pages (About, Contact, Recruitments)
6. Research pages (Office of R&D, Central Research Facility)
7. Announcements, news and events pages

The webpages were scraped and stored as a raw text file in the `data/raw/` directory.

2.2 Data Preprocessing

Raw scraped information tends to have noise like navigation text, links and formatting objects. Therefore, Python was used to implement a preprocessing pipeline.
The preprocessing steps include:

- Eliminating non-English text (such as Devanagari script)
- Eliminating URLs and email addresses
- Eliminating navigation menus and boilerplate text
- Converting all text to lowercase
- Sentence segmentation
- Word tokenization
- Eliminating punctuation-only tokens
- Eliminating very short words

The final corpus goes through preprocessing whereby, one sentence is put on a line and the tokens are separated by space.

The cleaned corpus is stored in:

➔ `data/cleaned_corpus.txt`

2.3 Dataset Statistics

The final processed dataset contains the following statistics:

DATASET STATISTICS — IIT JODHPUR CORPUS

Metric	Value
Total Documents	7
Total Sentences	1,133
Total Tokens	22,114
Vocabulary Size	4,074
Average Sentence Length	19.52 tokens

The 20 most frequent words in the dataset include:

Rank	Word	Frequency
1	student	262
2	institute	254
3	jodhpur	252
4	professor	251
5	engineering	225
6	research	221

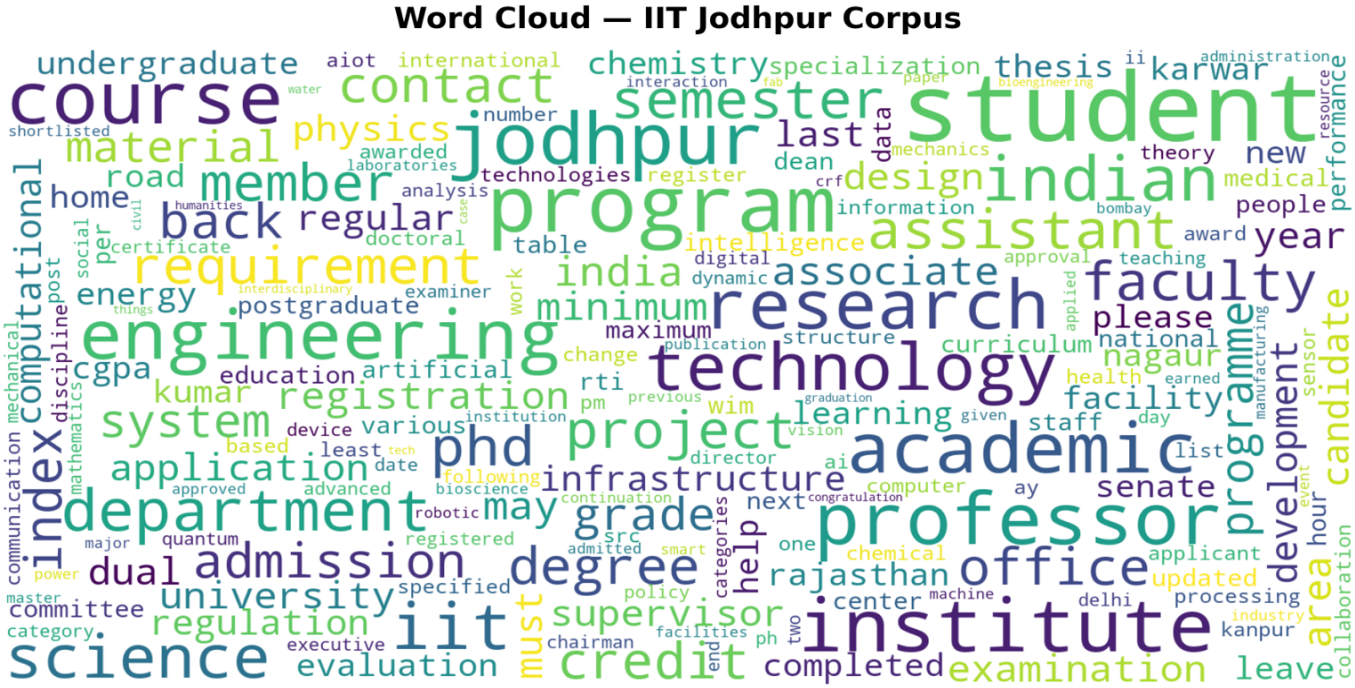
Rank	Word	Frequency
7	students	207
8	technology	203
9	iit	191
10	indian	186
11	program	184
12	academic	174
13	course	149
14	science	144
15	department	139
16	assistant	139
17	phd	138
18	faculty	135
19	programs	120
20	semester	117

These words reflect the academic context of the IIT Jodhpur corpus.

2.4 Word Cloud

A word cloud was generated to visualize the most frequent words appearing in the corpus.

Figure 1: Word Cloud showing frequently occurring words in the IIT Jodhpur dataset.



3. Model Training

To learn word embeddings, two Word2Vec architectures were implemented and trained.

3.1 Word2Vec Architectures

1. CBOW (Continuous Bag of Words)

CBOW predicts a target word based on the surrounding context words. The model takes the average of context word vectors and predicts the center word.

Characteristics:

- Faster training
- Works well for smaller datasets

2. Skip-gram with Negative Sampling

Skip-gram predicts surrounding context words given a target word.

Characteristics:

- Works better for rare words
- Captures semantic relationships more effectively
- Slightly slower training

3.2 Implementation Approaches

Two implementations were used:

1. **Scratch Implementation**
    - Implemented using NumPy
    - Includes manual negative sampling
    - Helps understand internal Word2Vec mechanics
2. **Gensim Implementation**
    - Uses optimized library implementation
    - Faster training
    - Serves as a benchmark

3.3 Hyperparameters

Multiple hyperparameter configurations were tested during training.

Parameter	Values Tested
Embedding Dimension	50, 100, 200
Window Size	3, 5, 7
Negative Samples	5, 10, 15
Epochs	10

Training was run for 10 epochs over the corpus. Words appearing fewer than two times were ignored to reduce noise in the vocabulary.

3.4 Implementation Details (Scratch Implementation)

The custom implementation (Word2VecScratch) was built using NumPy to manage the embedding weight matrices Win and Wout. The main techniques used in the implementation are:

- **Negative Sampling**: Instead of computing the full softmax over the entire vocabulary, negative sampling was used with a unigram distribution raised to the 3/4 power as proposed by *Mikolov et al. (2013)*.
- **Learning Rate Decay**: A linear scheduler was implemented to gradually decrease the learning rate from 0.025 to 0.0001 during training.
- **Numerical Stability**: Sigmoid inputs were clipped to the range [-10, 10] to avoid overflow issues in np.exp().
- **Dynamic Context Window**: Context window sizes were randomly reduced during training to generate more diverse context–target pairs.

3.5 Saved Models

The trained models generated during experimentation were stored inside the models/ directory. Both scratch implementations and Gensim models were saved using filenames that encode the architecture and hyperparameters used during training.

Examples include:

```
models/cbow_best.model
models/skipgram_best.model
models/cbow_best_scratch.pkl
models/skipgram_best_scratch.pkl
```

These models were later used for semantic analysis and visualization experiments.

3.6 Hyperparameter Experiments

To analyze the effect of different training configurations, multiple combinations of embedding dimensions, window sizes, and negative sampling values were tested.
The results of these experiments were stored in:

➔ `outputs/hyperparameter_results.csv`

Each configuration was evaluated using average cosine similarity across selected query words.

Key observations include:

- Embedding dimension in semantic similarity, raising embedded dimension slightly enhanced the results (progressively, at 50, 100, 150, etc.).
- The larger context windows were useful in capturing larger contextual relationships.
- Increased negative samples enhanced stability but training also increased.

The most efficient models that had been selected based on these experiments were stored to be visualized and analyzed in terms of semantics.

4. Semantic Analysis

Semantic similarity and analogy experiments were done to determine the quality of the learned word embeddings. The experiments are used to establish whether the trained models represent substantial relationships among words in the academic corpus gathered by the sources of IIT Jodhpur.

Two models were evaluated:

- **Skip-gram (Scratch Implementation)**
- **Skip-gram (Gensim Implementation)**

The results from both models were compared to analyze their ability to learn semantic relationships.

4.1 Evaluation Metric

To evaluate similarity between word embeddings, **cosine similarity** was used. Cosine similarity measures the angular distance between two embedding vectors and is defined as:

$$\cos(A, B) = \frac{A \,.\, B}{||A|| * ||B||}$$

where:

- A and B are word vectors
- A ·B is the dot product
- ||A|| and ||B|| are vector magnitudes

A similarity value closer to **1** indicates that the two words occur in similar contexts and therefore share similar semantic meaning.

4.2 Nearest Neighbour Analysis

In this experiment, Cosine similarity was used to retrieve the top-5 similar words of a query word. This assists to find out whether semantically related words are nearby in the embedding space.

Query Word: "research"

Rank	Scratch CBOW Word	Similarity	Gensim CBOW Word	Similarity
1	projects	0.9993	alumni	0.9895
2	publication	0.9992	home	0.9891
3	project	0.9992	faqs	0.9885
4	ongoing	0.9991	area	0.9883
5	collaborations	0.9989	applicants	0.9871

Observation

The embeddings also record significant terms related to research e.g. projects, publication and collaborations. The use of these words is typically related to the process of academic research, which means that the model manages to extract the contextual relationships based on the dataset and learns them.

Query Word: "student"

Rank	Scratch CBOW Word	Similarity	Gensim CBOW Word	Similarity
1	academic	0.9990	course	0.9978
2	program	0.9987	regular	0.9975
3	course	0.9967	maximum	0.9972
4	minimum	0.9936	register	0.9971
5	students	0.9932	categories	0.9965

Observation

The words used like academic, program, and course are similar to the word student indicating students have robust connections with academic programs. This indicates that the embeddings are effective in academic context.

Query Word: "phd"

Rank	Scratch Skip-gram Word	Similarity	Gensim Skip-gram Word	Similarity
1	selected	0.8254	news	0.8812
2	provisionally	0.7822	professor	0.8656
3	paul	0.7819	selected	0.8618
4	sme	0.7793	bsbe	0.8495
5	singh	0.7670	sme	0.8451

Observation

The nearest neighbours are words that contain academic announcements and faculty members like professor and selected. This states that the embeddings are contextual in nature, in as much as they relate to PhD admissions and research settings.

Query Word: "exam (examination)"

Rank	Scratch Skip-gram Word	Similarity	Gensim Skip-gram Word	Similarity
1	submission	0.9438	comprehensive	0.9741
2	report	0.9179	report	0.9529
3	satisfactory	0.9174	submission	0.9428
4	day	0.9169	examiners	0.9377
5	letter	0.9159	examiner	0.9289

Observation

Such words as submission, report and examiners are closely related to examination processes. This demonstrates that the embeddings are able to learn procedural and academic relationships in the data.

4.3 Analogy Experiments

Word embeddings are able to use simple vector arithmetic to learn semantic relations among words.

Example analogy structure:

vector(BTech)−vector(UG)+vector(PG)

The ideal result to such a relation would be that of a postgraduate, which is Mtech and the equivalent of BTech. A number of analogy tests have been conducted to test the presence of such relationships in the embeddings.

Analogy 1

UG : BTech :: PG : ?

Model	Predicted Word	Similarity
Skip-gram (Scratch)	mtech	0.785
Skip-gram (Scratch)	msc	0.751
Skip-gram (Gensim)	mtech	0.838
Skip-gram (Gensim)	msc	0.812

Comment

The two models are effective in terms of capturing the hierarchy of the academic program. The embeddings get the postgraduate equivalent of BTech, Mtech, right, which means that the model learns the academic relationships that are meaningful.

Analogy 2

Student : Study :: Faculty : ?

Model	Predicted Word	Similarity
Skip-gram (Scratch)	positions	0.839
Skip-gram (Scratch)	post	0.837
Skip-gram (Gensim)	people	0.804
Skip-gram (Gensim)	members	0.796

Comment

The hypothesized words are an indication that there are links between the faculty members and the roles in the institutions. The words like position, post, and members are administrative and organizational in nature with regard to the faculty.

Analogy 3

Department : HOD :: Institute : ?

Model	Result
Skip-gram (Scratch)	Word Missing
Skip-gram (Gensim)	Word Missing

Comment

The term HOD (Head of Department) was not used very often in the dataset to satisfy the min_count requirement. Thus, the model was not able to calculate the analogy as the word was not in the vocabulary.

Analogy 4

BTech : Undergraduate :: PhD : ?

Model	Predicted Word	Similarity
Skip-gram (Scratch)	admissions	0.729
Skip-gram (Scratch)	postgraduate	0.702
Skip-gram (Gensim)	philosophy	0.731
Skip-gram (Gensim)	postgraduate	0.724

Comment

The two models access postgraduate, which is the rightful academic level that is related to doctoral studies. This shows that the embeddings have hierarchical academic relationships.

4.4 Overall Observations

The analogy experiments show that the embeddings capture meaningful semantic relationships from the academic corpus. Both implementations successfully identify relationships such as BTech–MTech for undergraduate and postgraduate programs, indicating that the models learn academic hierarchies.

However, some limitations are observed due to:

- Small corpus size
- Domain-specific vocabulary
- Certain words not meeting the min_count threshold (e.g., *HOD*)

Despite these limitations, the embeddings demonstrate the ability to capture academic context and semantic associations, with the Gensim model generally producing slightly higher similarity scores.

5. Visualization of Word Embeddings

In order to figure out more about the organization of the learned word embeddings, the high dimensional vectors were mapped into two-dimensional space to visualize. The dimensionality reduction aids in revealing the clusters of semantically related words and we can examine the extent in which the model succeeds in identifying relationships within the dataset.

Two dimensionality reduction methods that were entailed frequently included:

- **Principal Component Analysis (PCA)** – a linear dimensionality reduction method that projects data onto directions of maximum variance.
- **t-Distributed Stochastic Neighbor Embedding (t-SNE)** – a nonlinear technique designed to preserve local relationships between data points.

These techniques were applied to embeddings generated from both:

- CBOW models
- Skip-gram models

Each of these was trained using:

- Scratch implementation
- Gensim implementation

5.1 PCA Visualization

Principal Component Analysis (PCA) reduces the high-dimensional embedding vectors to two principal components that capture the largest variance in the dataset. This allows us to observe the global structure of word relationships and identify clusters formed by semantically similar words. The PCA projections for different models are shown below.

PCA – CBOW (Scratch Implementation)

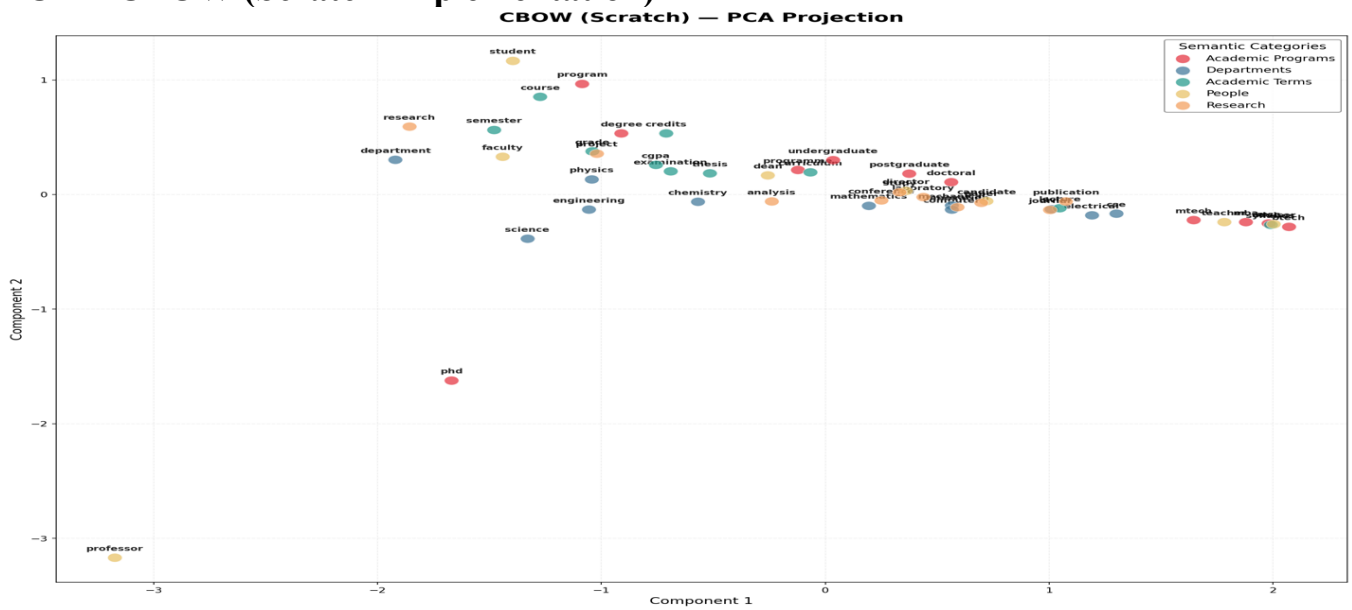


Figure 2: PCA projection of word embeddings generated using the CBOW model implemented from scratch.

PCA – CBOW (Gensim)

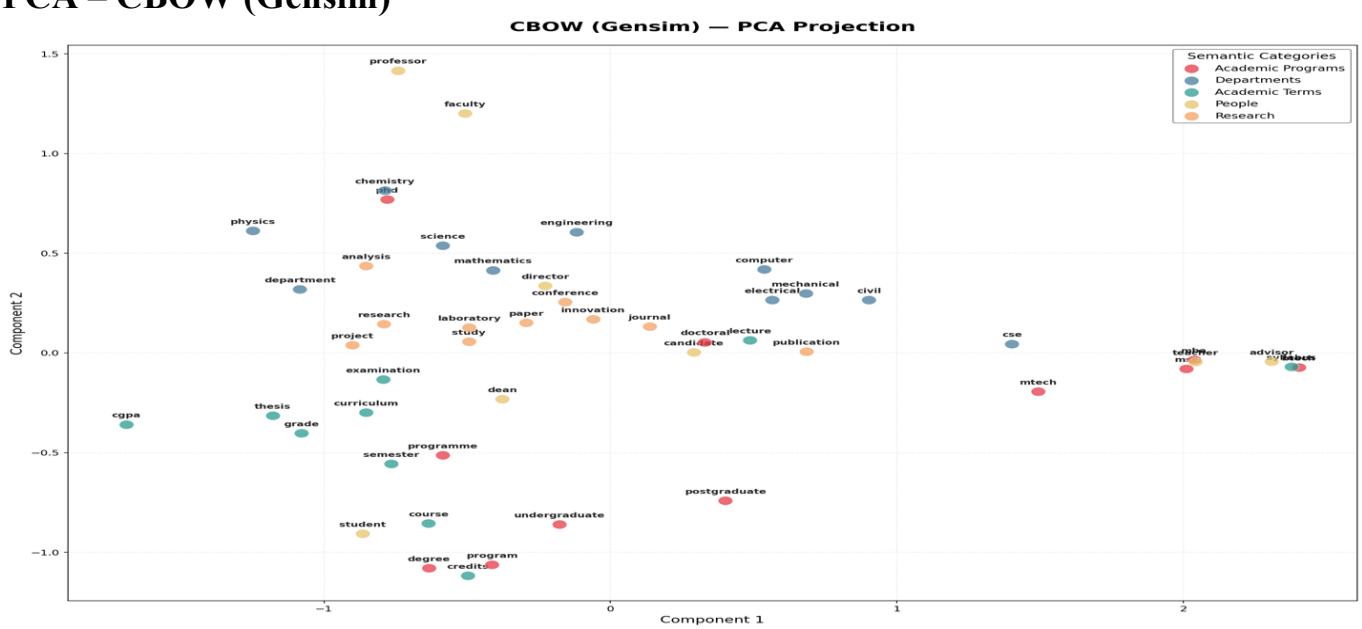


Figure 3: PCA projection of word embeddings generated using the CBOW model trained with the Gensim library.

PCA – Skip-gram (Scratch Implementation)

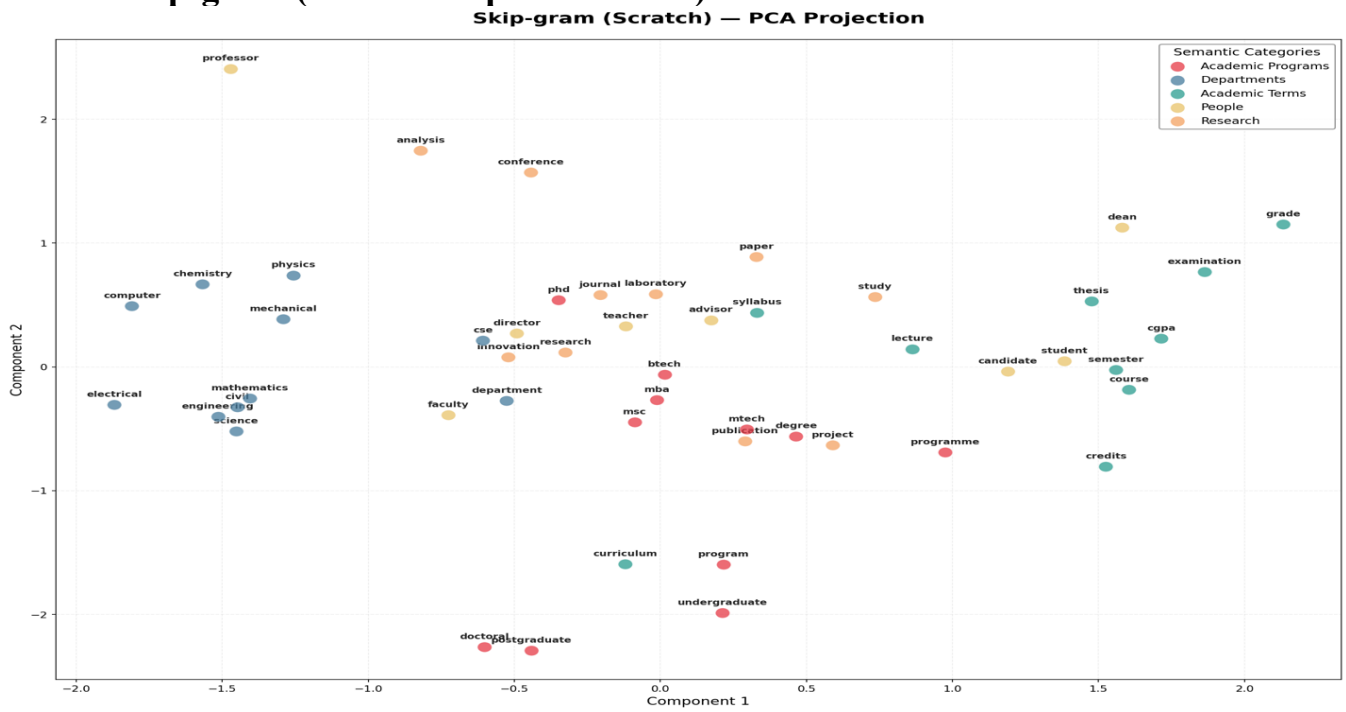


Figure 4: PCA projection of embeddings generated using the Skip-gram model implemented from scratch.

PCA – Skip-gram (Gensim)

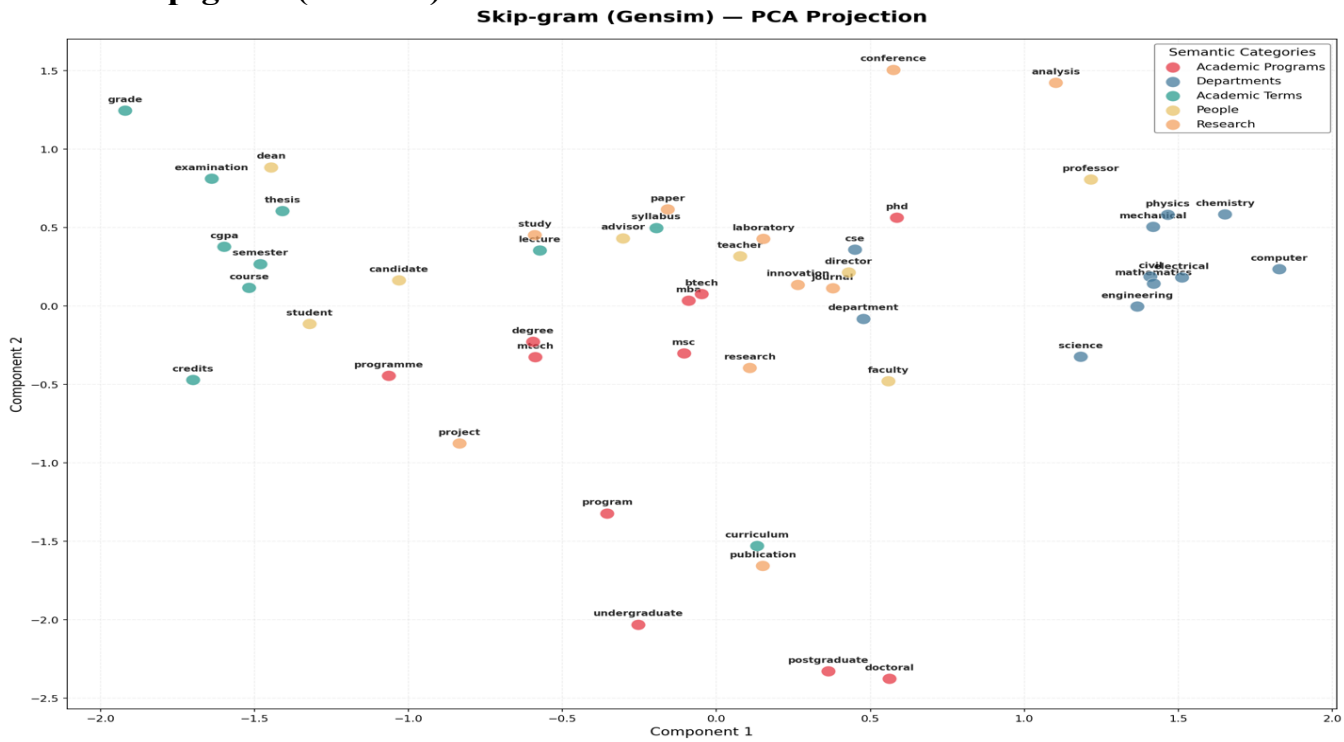


Figure 5: PCA projection of embeddings generated using the Skip-gram model trained using Gensim.

5.2 t-SNE Visualization

While PCA preserves global structure, t-SNE is more effective at revealing local clusters and fine-grained semantic relationships. It is particularly useful for visualizing high-dimensional embedding spaces, where related words tend to appear closer together.
The t-SNE visualizations for the different models are presented below.

t-SNE – CBOW (Scratch Implementation)

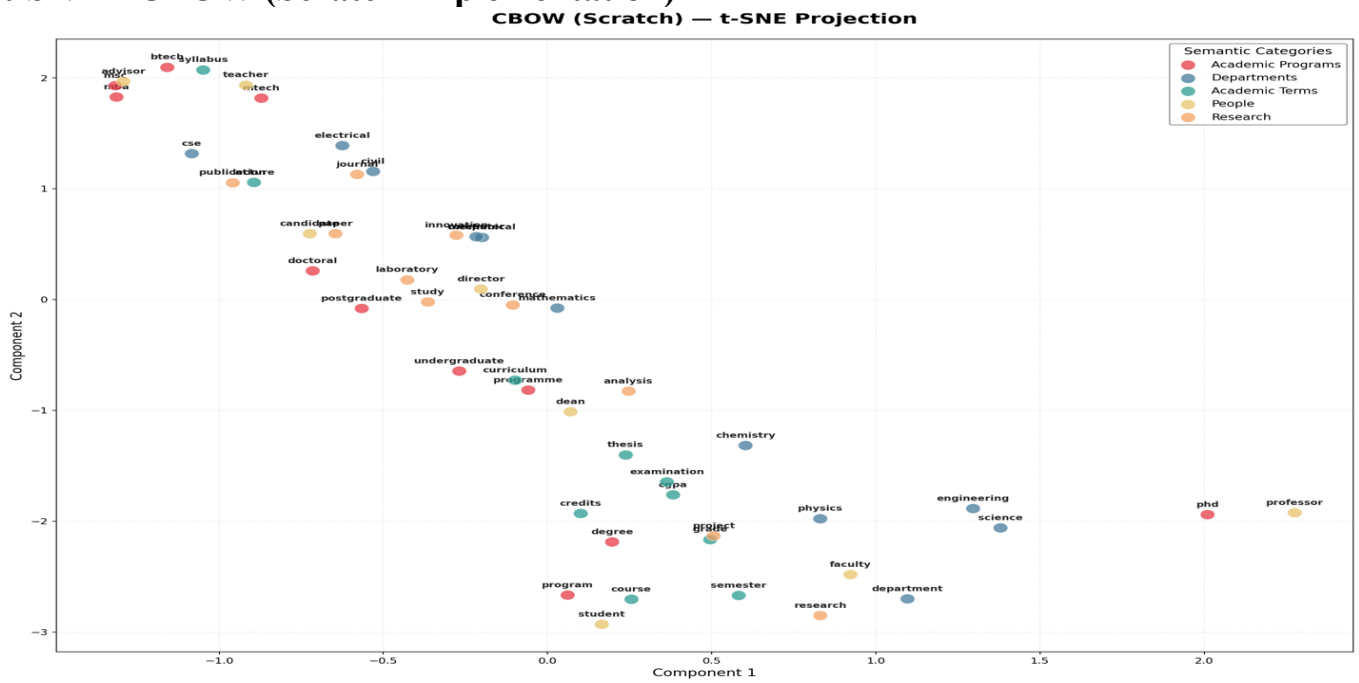


Figure 6: t-SNE projection of embeddings produced by the CBOW model implemented from scratch.

t-SNE – CBOW (Gensim)

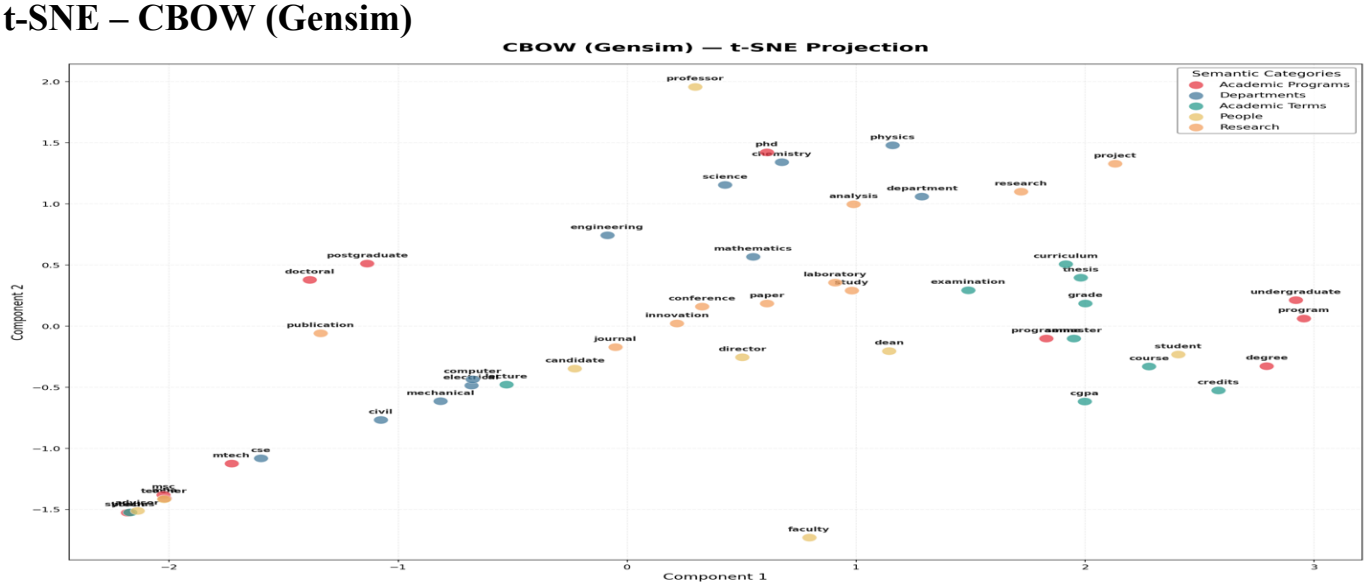


Figure 7: t-SNE projection of embeddings produced by the CBOW model trained using Gensim.

t-SNE – Skip-gram (Scratch Implementation)

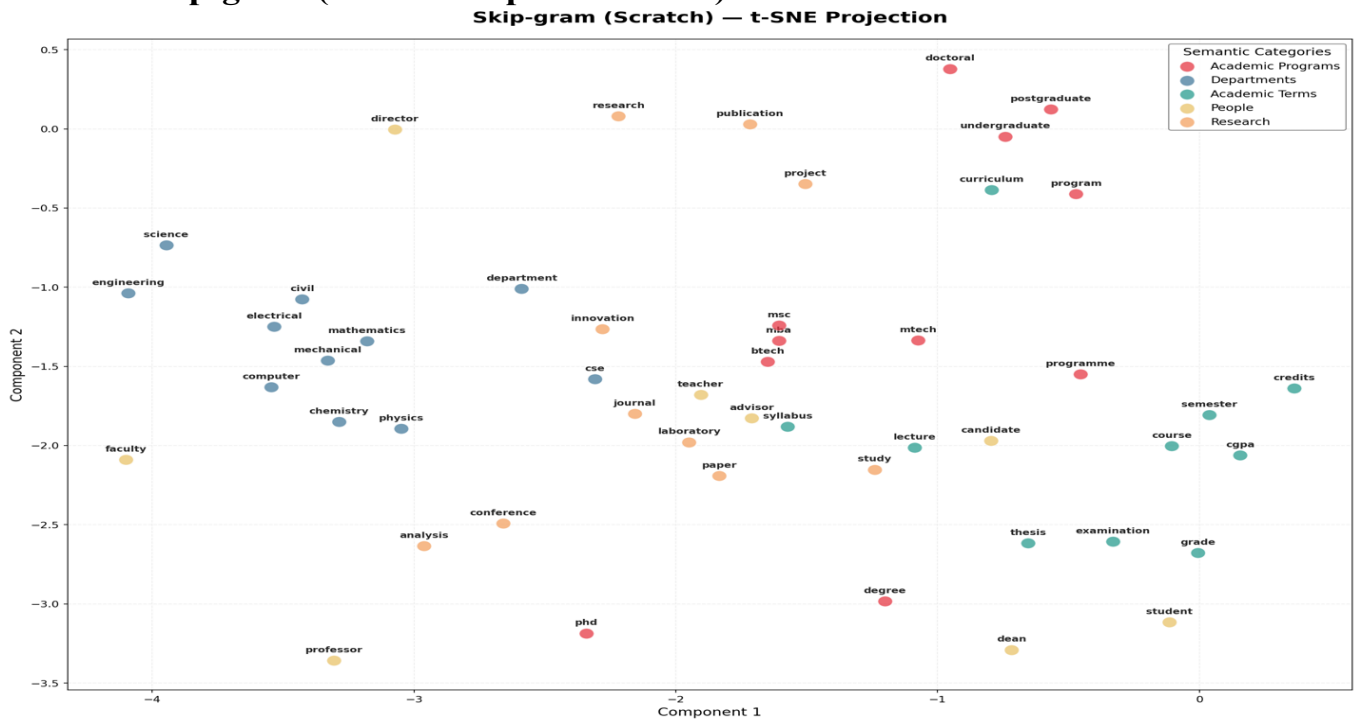


Figure 8: t-SNE projection of embeddings generated by the Skip-gram model implemented from scratch.

t-SNE – Skip-gram (Gensim)

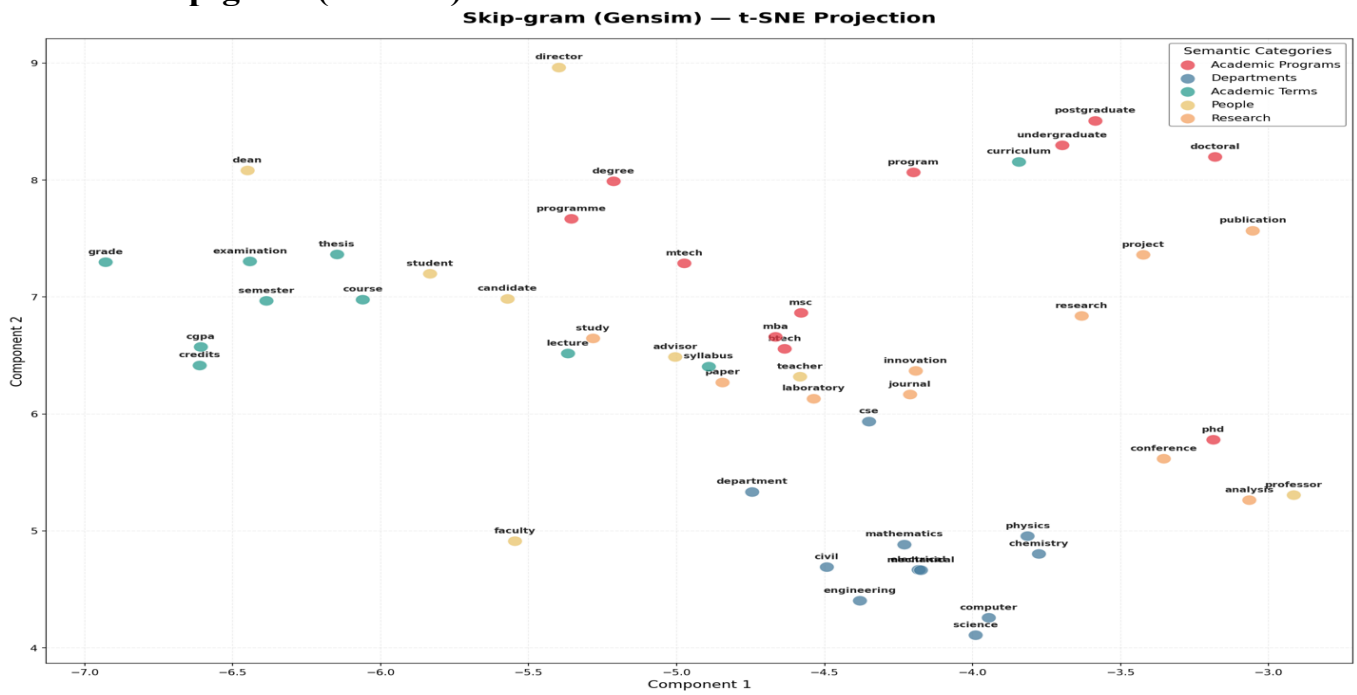


Figure 9: t-SNE projection of embeddings generated by the Skip-gram model trained using Gensim.

5.3 Training Performance Curves

To analyze the training behaviour and computational efficiency of the models, additional plots were generated to track the training loss over time and compare the training time between implementations. The metrics are used to determine the speed at which the models would converge and the efficiency with which the process of training will be implemented.

Training Loss Curves

The loss that was experienced in the training phase was recorded and plotted to ensure the convergence was observed in training.

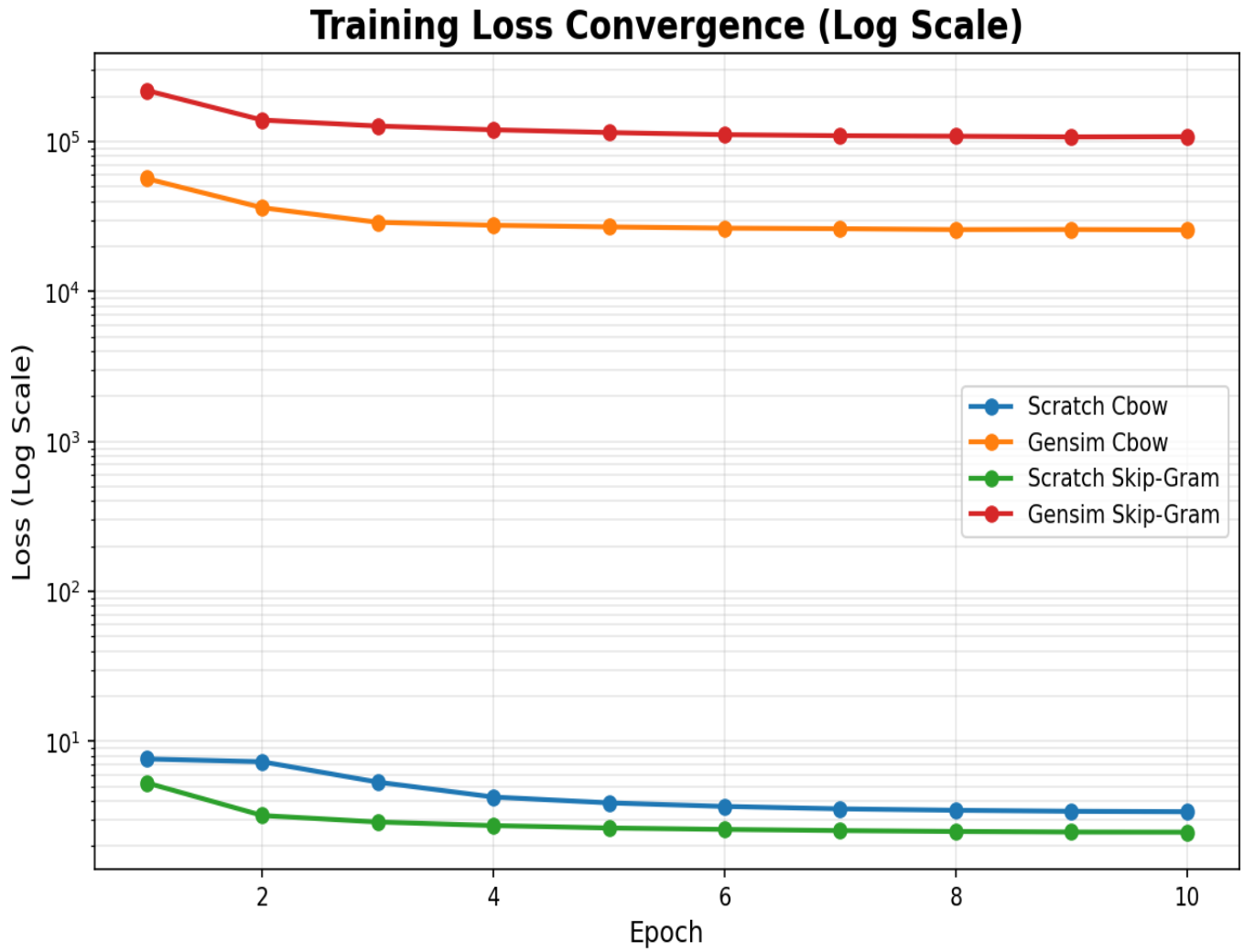


Figure 10: Training loss curves showing convergence behaviour during Word2Vec training.

Observation:

- The loss reduces gradually with the increase in training, which suggests that the model is able to learn meaningful word representations.
- The loss decrease is slow, indicating the stable optimization with the negative sampling through stochastic gradient descent.
- There are no large oscillations present, which shows that the learning rate schedule was suitable in the training process.

Training Time Comparison

The total training time required by the scratch implementation and the Gensim implementation was also compared.

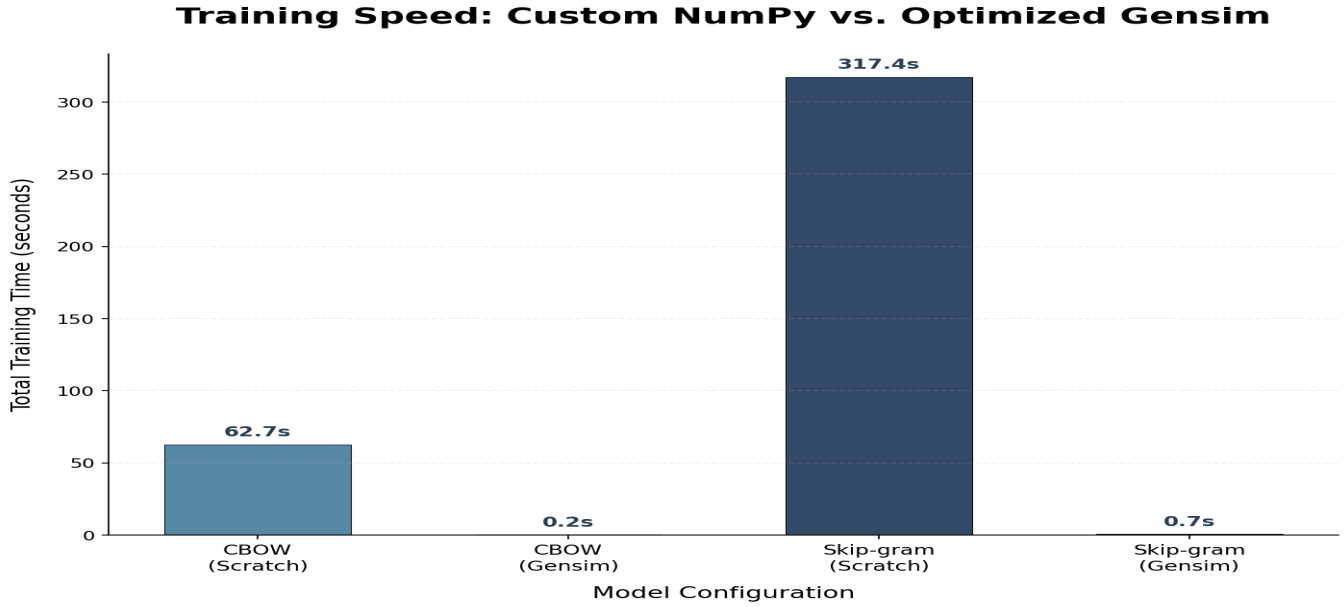


Figure 11: Comparison of training time between the scratch implementation and the optimized Gensim implementation.

Observation:

- Implementation of the scratch is much slower with Python loops and manual gradient calculation.
- Gensim has been far more optimized, since the low level routines are in C which are optimized.
- This analogy points out the benefit of optimization of libraries in case of working on a large scale of text data.

5.4 Interpretation

It is possible to note several semantic clusters in the PCA and t-SNE plots.

- **Academic Programs:** the words btech, mtech and phd are found near to each other showing their connection as the academic degree programs.
- **Departments:** such words as cse, ee, and me compose another cluster, which denotes academic departments.
- **Academic Roles:** the terms like professor, student, and faculty are united with each other, which means that they have some similar roles in the university system.

Comprehensively, the learned embeddings verify that the learned embeddings are substantial semantic relationships. Embeddings generated by skip-gram tend to form more compact and better-distinct clusters whereas embeddings generated by CBOW have a more dispersed look.

6. Discussion

The scratch models were able to learn meaningful word embeddings that were similar to those produced by Gensim.

Key observations:

- For the nearest neighbours and analogy result, the scratch models gave similar results as the Gensim models.
- Training times were also substantially more in the case of scratch implementations (approximately 60 seconds in the case of CBOW) than less than a second in the case of Gensim, because of optimized C-based implementations.
- During the training the training loss curves of both implementations converge and stabilize.
- An increase in the size of the window slightly enhanced contextual similarity of related academic terms.
- Results of visualization indicate that the two implementations create meaningful clusters of academic and institutional terms.

This task has given me a positive understanding of how Word2Vec trains semantic relationships through vectors representations as well as negative sampling.

7. Conclusion

This project illustrated the entire process of learning word representations using a self-constructed scholarly dataset.

The main steps involved:

- Web scraping: Scraping removes textual information about IIT Jodhpur.
- Preprocessing of data such as cleaning, tokenization and vocabulary generation.
- What Word2Vec models can be trained. How to train Word2Vec models with both scratch and Gensim library.
- A semantic evaluation based on nearest neighbour and analogy experiments.
- The PCA and t-SNE are used to visualize embeddings and examine their patterns in clustering.

The scratch implementations as well as the Gensim models were effective enough to retrieve meaningful semantic relationships among academic words. Although the scratch models were useful in comprehending the internal mechanics of Word2Vec, the Gensim implementation was much more useful since it allowed much faster training because of optimized libraries.

On the whole, the given assignment was an enlightening experience about the learning of word representations, semantics in vectors, and trade-offs, in practice, between writing algorithms by hand and using efficient machine learning libraries.